

AI Tools, Agents & Your Stack

A plain-English briefing — turning “am I behind?” into a teachable concept for Build-a-Stack
Build-a-Stack companion doc · Phase A-3 candidate material · Drafted June 2026

The one-sentence takeaway: You are not behind. You already run a serious automation stack. The thing the industry is buzzing about is a *different kind of brain* — a reasoning agent — that can plug into the body you’ve already built. Whether to use one is an engineering tradeoff, not a race to catch up.

1 · Three tools everyone name-drops

They get lumped together, but they are three different categories. Sorting them out removes most of the “I feel behind” feeling.

Tool	What it actually is	Who is in control
Devin	An autonomous “AI software engineer” (a product). You hand it a ticket; it plans, codes, runs, and opens a pull request.	The AI
Cursor	An AI-powered code <i>editor</i> (a VS Code fork). A human writes code, supercharged by AI autocomplete, inline edits, and a codebase-aware chat.	You
LangGraph	A <i>library</i> (Python/JS) for <i>building</i> multi-step AI agents — loops, memory, branching. Not something you write code <i>in</i> ; it’s plumbing you build <i>with</i> .	N/A — a building block

Key split: Devin and Cursor *compete* over “help me write code faster.” LangGraph isn’t in that race at all — it’s a component for *making* AI software. And the agentic workflow you’re using right now (an AI committing and pushing to your repo) is the *same category* as Devin. You’re already doing it.

2 · The big idea: automation vs. a reasoning agent

This is the distinction worth internalizing. Everything else is detail.

What you have today — deterministic automation

Your bots are **rule-followers**. You wrote the logic ahead of time:

WHEN websocket price crosses X AND a TradingView alert fires → place trade. IF a position drifts → the supervisor bot flattens it.

The bot doesn’t *decide* in any human sense — it evaluates conditions you predefined and runs the matching branch. **Same inputs → same output, every time.** Your supervisor bots watching other bots are still rules (“if a child bot stops heart-beating, restart it”) — that pattern is a **watchdog**, already in your glossary. This is predictable, auditable, and — crucial for trading — **provable**.

What an AI agent adds — reasoning over a model

An LLM agent doesn't run your if/then. It **interprets a situation in natural language and chooses what to do**, including things you never explicitly coded:

Observe (market data, news, positions) → reason in plain language about what's happening → pick an action from the tools you gave it (place trade, resize, alert you, do nothing) → see the result → reason again.

Your bots	An AI agent
Execute rules you wrote	Decides what to do
Same input → same output	Same input → <i>maybe different</i> output
You can prove its behavior	You can only observe its behavior
"Cross X → buy"	"Price crossed X, but volume is thin and there's a Fed headline — I'll wait"

That last row is the entire point: an agent can weigh **fuzzy, unstructured information** (a news headline, an ambiguous pattern) and handle situations **you didn't anticipate** — things you can't easily encode as if/then.

3 · “But wasn’t I already doing this?”

Partly — and it's the hard part. Websockets, TradingView alerts, execution, supervisors: that's the *plumbing* an agent also needs. **You've built the body.** An AI agent is just a *different brain* plugged into that same body.

The piece you don't have is a non-deterministic LLM sitting in the decision seat instead of your hand-written rules. That brain — its multi-step reasoning loop, memory, and branching — is exactly what a library like **LangGraph** helps you build.

4 · The honest warning (this matters for trading)

An LLM agent is non-deterministic. Same market, run it twice, it might do two different things. In a chatbot that's fine. On a path that **places real trades with real money**, that property is dangerous:

- You can't fully predict it.
- It can “hallucinate” a reason and act on it.
- Backtesting is murky — past behavior doesn't guarantee future behavior.
- Latency: an LLM “thinks” in seconds; your websocket bots react in milliseconds.

The mature pattern

Not “replace my bots with an AI agent.” Instead:

- **Keep deterministic bots on the trigger/execution path** — fast, provable, safe.
- **Put the AI agent in an advisory / supervisory seat** — it watches, reasons over the fuzzy stuff (news, regime shifts, “does this setup still make sense?”), and either *alerts you* or *adjusts parameters within hard guardrails*. The actual trade trigger stays rule-based, with circuit breakers the agent **cannot** override.

That buys you the agent's judgment **without** handing a slot machine your order-entry.

5 · Your plan: the agent as a “cube entity”

Decision: if an LLM agent is added, it will be a **cube entity** — a class of agent in the stack that **never places trades**. It observes, reasons, and reports.

This is the **safest, most measurable** way to test whether agent reasoning is actually effective: it sits beside the proven deterministic path, its calls can be logged and compared against what actually happened, and a bad call costs *nothing* because it has no authority to execute. You get a clean read on signal quality before ever granting it more responsibility.

This maps onto the “know your debt” philosophy Build-a-Stack already teaches: adopt the new capability deliberately, in its safest form, with a way to measure the tradeoff.

6 · So, which tools should you actually care about?

Tool	Verdict for you
Cursor	Optional convenience. A nicer local editor, but a sidegrade to the Claude Code workflow you already use. Not a gap.
Devin	Not needed. You already run an equivalent agentic coding workflow. Swapping tools, not gaining a capability.
LangGraph	Only relevant <i>if/when</i> you build the cube-entity advisory agent. Then it's the right kind of building block for the reasoning loop.

7 · How this folds into Build-a-Stack

Build-a-Stack already teaches the plumbing — ports, processes, websockets, watchdogs, and **MCP** (the very protocol this whole agent world runs on). The natural next teaching unit:

- **Glossary card — “Rule-bot vs. AI agent”:** same body, different brain; deterministic vs. reasoning; provable vs. observable — taught in the existing three layers (what it is → how the real stack uses it → pragmatic vs. production tradeoff).
- **Glossary card — “Cube entity / advisory agent”:** the guardrail pattern — an agent that reasons but cannot execute, and why that’s the safe, measurable way to adopt one.
- **Build challenge:** wire an advisory agent that observes the data feed and emits an alert — but is physically prevented from connecting to the execution path.

Build-a-Stack — an interactive way to learn how real software systems connect, grounded in a real trading-operations stack. Taught in three honest layers: what it is → how the real stack uses it → how a production-grade version would differ, and why. This companion document is candidate source material for the Phase A-3 glossary and a future build challenge.